

DOTS Sprite Animator

User Guide · v0.8.1

Invert Lab · `com.invertlab.spriteanimator`

Portable DOTS-first 2D flipbook animation with timeline authoring, events, sockets, Query/Unity2D colliders, and optional Unity Physics. Includes **ECS system run order** inside Unity's default groups so you know where to place your own systems.

Contents

1. Quick start
2. Architecture
3. System run order (Unity + Invert Lab + Physics)
4. Colliders & Unity Physics
5. PlaybackPath / GPU limits
6. Samples & Public API notes

1. Quick start

Open **Window** → **DOTS Sprite Animator**, assign a sheet, slice or set Columns×Rows, author clips / frames / events / sockets / colliders, save a profile asset, then add `SpriteAnimSetAuthoring` (and optional `SpriteAnimPlayerAuthoring`) on an entity under a `SubScene`.

See also `Documentation~/QuickStart.md` in the package.

2. Architecture

Profile (Window > DOTS Sprite Animator)
-> clips / frames / TRS / events / sockets / collider boxes
`SpriteAnimSetAuthoring` (+ optional `SpriteAnimPlayerAuthoring`)
-> bake (SubScene) or hybrid preview
Play (CPU player, or GPU for simple flipbooks)
-> `SpriteInstance` / GPU render

Animation works with **no** collider method and **no** Unity Physics package.

Runtime folders

Folder	Contents
Authoring/	MonoBehaviour bakers (*Authoring)
Components/	<code>IComponentData</code> / buffers
Systems/	<code>ISystem</code> / <code>SystemBase</code> playback & events
Profile/	Sheet profile + helpers
Utility/	Factories, shaders, playback helpers
Feature areas	Instanced, Hitboxes, Sorting, Spawn, Culling, <code>PureOverlap</code> , <code>UnityPhysics</code> — nested Authoring/Components/Systems/...

Naming: `FooAuthoring`, `FooSystem`, data as `Foo` (use `FooData` only if the short name collides).

3. System run order (Unity + Invert Lab + Physics)

Attributes in code are the source of truth: [UpdateInGroup], [UpdateBefore], [UpdateAfter], OrderFirst, OrderLast.

3.1 Unity default frame (where groups sit)

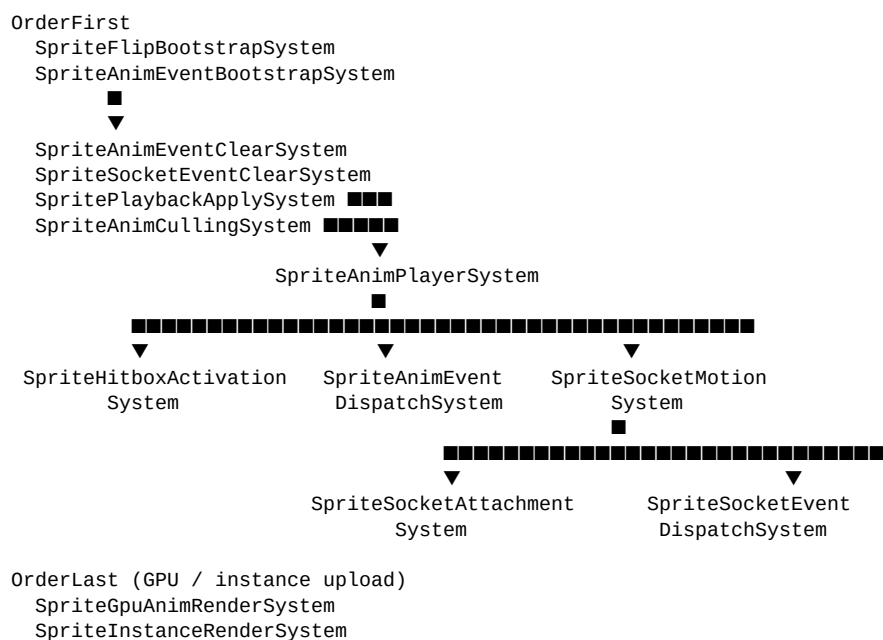
```

InitializationSystemGroup
SimulationSystemGroup      ← most Invert Lab animation / gameplay
  · early / OrderFirst systems
  · TransformSystemGroup
  · late / OrderLast systems
FixedStepSimulationSystemGroup ← Unity Physics (fixed timestep)
  ■■ PhysicsSystemGroup
PresentationSystemGroup

```

Invert Lab animation clocks run in **SimulationSystemGroup** (variable step). Unity Physics (optional) runs under **FixedStepSimulationSystemGroup** → **PhysicsSystemGroup**. Those are **different clocks**.

3.2 Invert Lab — SimulationSystemGroup chain



Also in Simulation (weaker / no hard edge to Player)

System	Notes
SpriteSortDepthSystem	UpdateBefore(TransformSystemGroup)
SpriteTintTweenSystem	Simulation only
SpriteClipSheetSystem	Simulation only
SpriteSheetRegistrationSystem	Simulation only
SpriteBatchPlaceDriver / SpriteCrowdGpuClipDriver	Managed helpers, default order

Baking (not play loop): SpriteAnimSetPreviewRenderStripBakingSystem → PostBakingSystemGroup.

3.3 Unity Physics — FixedUpdate (optional com.unity.physics)

```
FixedStepSimulationSystemGroup
  ■■ PhysicsSystemGroup
```

- ■ BeforePhysicsSystemGroup ← write velocities / forces before build
- ■ PhysicsInitializeGroup ← BuildPhysicsWorld
- ■ PhysicsSimulationGroup ← broadphase → contacts → solve
- ■ ExportPhysicsWorld
- ■ AfterPhysicsSystemGroup ← OverlapAabb / read PhysicsWorldSingleton
- ■ UnityPhysicsOverlapBridge ← Invert Lab (pinned here)

UnityPhysicsOverlapBridge is in **AfterPhysicsSystemGroup** (after ExportPhysicsWorld) so OverlapAabb sees the rebuilt world.

Attack flow: (1) Simulation — player advances, hitboxes activate. (2) Gameplay reads AABB via SpriteHitboxQuery and calls UnityPhysicsOverlapBridge.Queue(...). (3) FixedStep AfterPhysics — bridge OverlapAabb vs Character bodies.

3.4 Where to put YOUR systems

Animation / sockets / events (Simulation)

You want to...	Put your system...
Change playback before the clock ticks	UpdateBefore(SpriteAnimPlayerSystem) (usually after EventClear)
Read frame, events, or live hitboxes	UpdateAfter(SpriteAnimPlayerSystem); after HitboxActivation if you need active boxes
After Independent Motion sockets settle	UpdateAfter(SpriteSocketAttachmentSystem)
React to socket triggers	UpdateAfter(SpriteSocketEventDispatchSystem)
After our GPU packs	UpdateAfter(SpriteInstanceRenderSystem) or Presentation
Sort depth before LocalToWorld	UpdateBefore(TransformSystemGroup)

```
[UpdateInGroup(typeof(SimulationSystemGroup))]  
[UpdateAfter(typeof(SpriteHitboxActivationSystem))]  
public partial struct MyMeleeResolveSystem : ISystem { }
```

Unity Physics (FixedStep)

You want to...	Put your system...
Set velocities before the world builds	UpdateInGroup(BeforePhysicsSystemGroup)
Overlap / raycast the live world	UpdateInGroup(AfterPhysicsSystemGroup)
After our queued overlaps	AfterPhysicsSystemGroup + UpdateAfter(UnityPhysicsOverlapBridge)

```
using Unity.Physics.Systems;
```

```
[UpdateInGroup(typeof(AfterPhysicsSystemGroup))]  
[UpdateAfter(typeof(UnityPhysicsOverlapBridge))]  
public partial struct MyDamageApplySystem : ISystem { }
```

Query-only (no Physics package): stay in Simulation after SpriteHitboxActivationSystem and call SpriteHitboxQuery.

4. Colliders & Unity Physics

Method	What it does	Needs
Query	Frame/Clip/Character boxes as data; SpriteHitboxQuery AABB	nothing extra
Unity2D	Spawns Collider2D children	built-in 2D physics
UnityPhysics	Character body → Box/Sphere/Convex; frame stays AABB query	optional com.unity.physics

Lifetime: Character = persistent hurtbox (bake / Ensure). Frame/Clip = Query AABB, then optional OverlapAabb vs Character PhysicsCollider.

Attack frame AABB --OverlapAabb--> Character PhysicsCollider (hurtbox)

5. PlaybackPath / GPU limits

Auto = CPU default. PreferGpu = convert once when eligible. ForceCpu = never GPU.

GPU path is a simple flipbook (single sheet, uniform grid, no sockets/events/reorder/custom holds/per-frame TRS). PreferGpu for crowds; keep heroes on CPU when you need events or sockets.

6. Samples & Public API

Optional Samples~ via Package Manager Samples tab. Public notes: Documentation~/PublicAPI.md. Full detail: SystemRunOrder.md, Architecture.md.